Building a Serverless Crypto Data Pipeline using AWS Services and Power BI

1. Project Overview

This project demonstrates the development of a server-less cloud data pipeline using core AWS services and Power BI. The pipeline automates the collection of real-time cryptocurrency price data from a public API, stores it in an Amazon S3 data lake, and enables SQL querying via Amazon Athena for final visualization in Power BI.

The architecture was intentionally designed to replicate real-world data engineering workflows - focusing on ingestion, storage, querying, and visualization, while remaining entirely within the AWS Free Tier.

2. Project Objectives

- **Understand Cloud Data Pipelines:** Gain hands-on experience with modern cloud-based data workflows.
- **Master Server-less Interaction:** Learn how AWS server-less services (Lambda, S3, Athena) trigger and interact with one another securely.
- **End-to-End Execution:** Practice the full life cycle of data ingestion, storage, SQL querying, and business intelligence visualization.

3. Architecture & Technologies Used

The pipeline follows a highly decoupled, server-less architecture.

- **Data Source:** CoinGecko API
- **Compute / Ingestion:** AWS Lambda (Python 3.x, Boto3)
- **Data Storage:** Amazon S3
- **Analytics & Querying:** Amazon Athena
- **Data Visualization:** Microsoft Power BI
- **Security & Access:** AWS IAM (Execution Roles and ODBC Credentials)

4. Data Ingestion (AWS Lambda)

A Python script deployed in AWS Lambda fetches the current market price of Bitcoin and Ethereum from the CoinGecko API. The Lambda function triggers the HTTP request, parses the JSON response, formats it into a CSV structure, and uploads the file directly to an S3 bucket.

To ensure each execution produces a unique dataset without overwriting previous pulls, the function dynamically generates a timestamped filename

(e.g.crypto_YYYYMMDD_HHMMSS.csv).

Lambda Python Code:

```python
import json
import boto3
import urllib.request
from datetime import datetime

BUCKET = "adarsh-aws-project-bucket"
FOLDER = "c_data/"

def lambda_handler(event, context):

    url = "https://api.coingecko.com/api/v3/simple/price?ids=bitcoin,ethereum&vs_currencies=usd"

    response = urllib.request.urlopen(url)
    data = json.loads(response.read())

    timestamp = datetime.utcnow()

    csv_content = f"timestamp,bitcoin_usd,ethereum_usd\n{timestamp},{data['bitcoin']['usd']},{data['ethereum']['usd']}"

    filename = f"{FOLDER}crypto_{timestamp.strftime('%Y%m%d_%H%M%S')}.csv"

    s3 = boto3.client("s3")

    s3.put_object(
        Bucket=BUCKET,
        Key=filename,
        Body=csv_content
    )

    return {
        "statusCode": 200,
        "message": "File uploaded to S3"
    }
```

Example of the raw API JSON response before CSV conversion:

JSON

```
{
  "bitcoin": {"usd": 69944},
  "ethereum": {"usd": 2032}
}
```

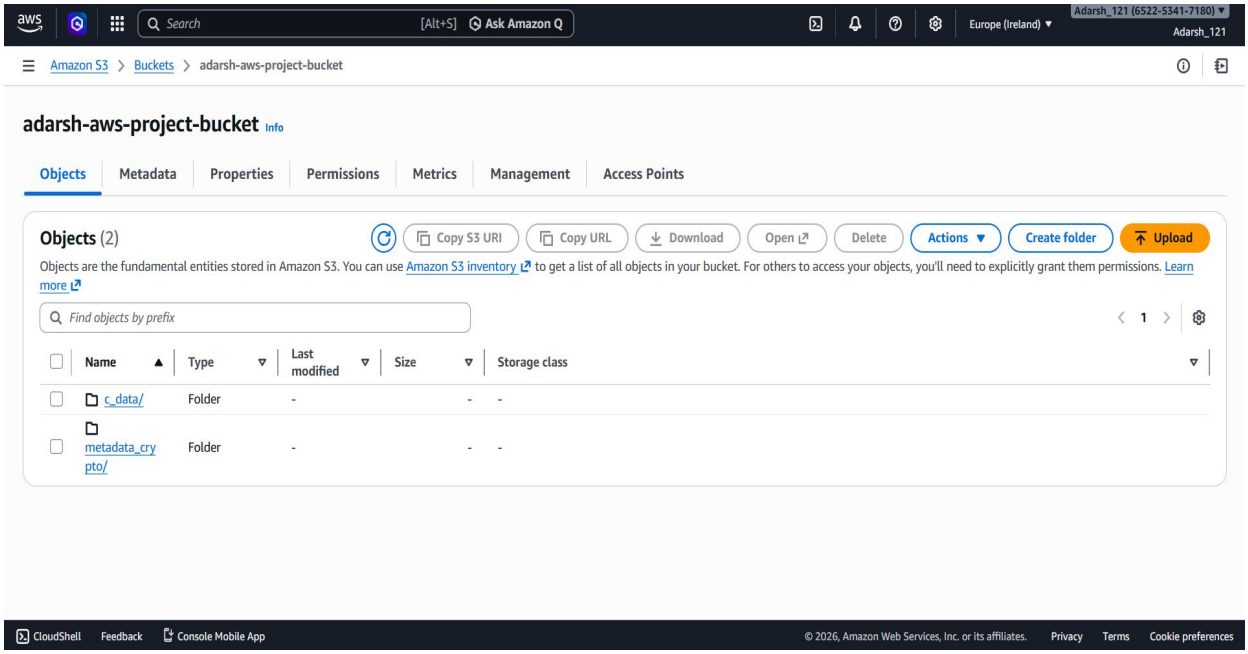
5. Data Storage (Amazon S3)

Amazon S3 acts as the data lake storage layer. Proper folder structuring was critical for the pipeline to function correctly with downstream querying tools. The bucket was organized to isolate the raw CSV data from temporary metadata files:

adarsh-aws-project-bucket/

-> c_data/ (Contains only the generated CSV file)

-> metadata_crypto/



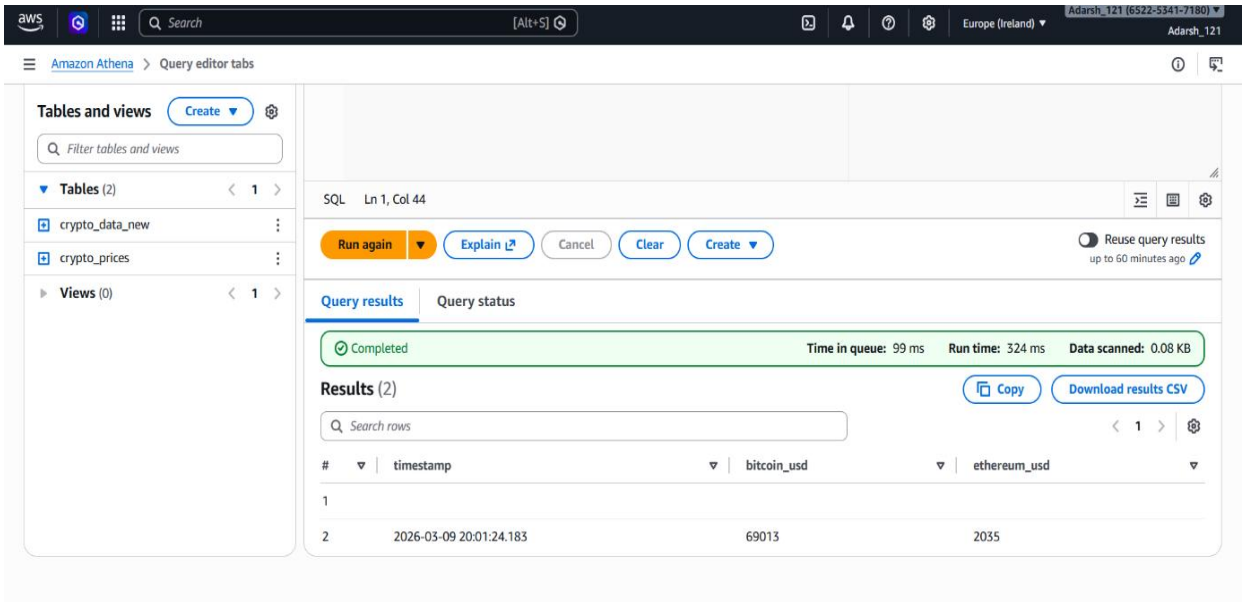
6. Querying Data (Amazon Athena)

Amazon Athena was used to query the stored CSV files directly from S3 using standard SQL, eliminating the need for a traditional relational database server.

Initial Setup: Initially, I created a base table to test the querying capabilities and verify that the data was readable.

Query:

*SELECT * FROM "crypto_db.crypto_prices";*

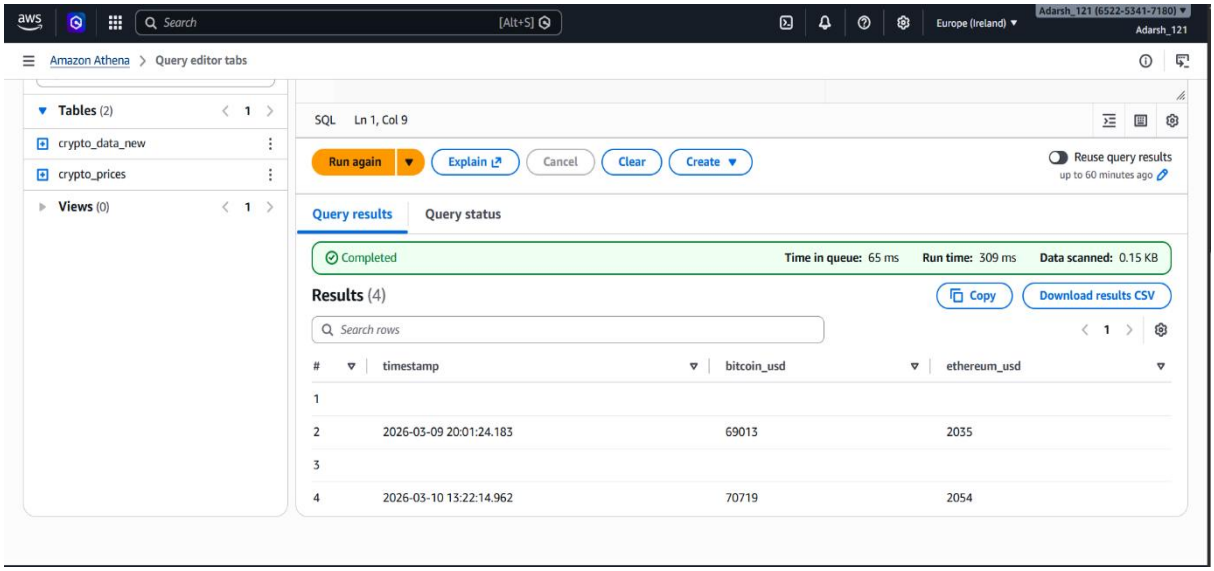


Handling Dynamic Data Updates: Because the AWS Lambda function was set up to trigger multiple times and drop new timestamped CSV files into S3 continuously, the architecture needed to adapt. I created a new external table, crypto_data_new, specifically configured to point to the isolated c_data/ folder.

This allowed Athena to automatically recognize and query any new files dropped into that folder by Lambda without needing to rebuild the table each time.

Query Structure:

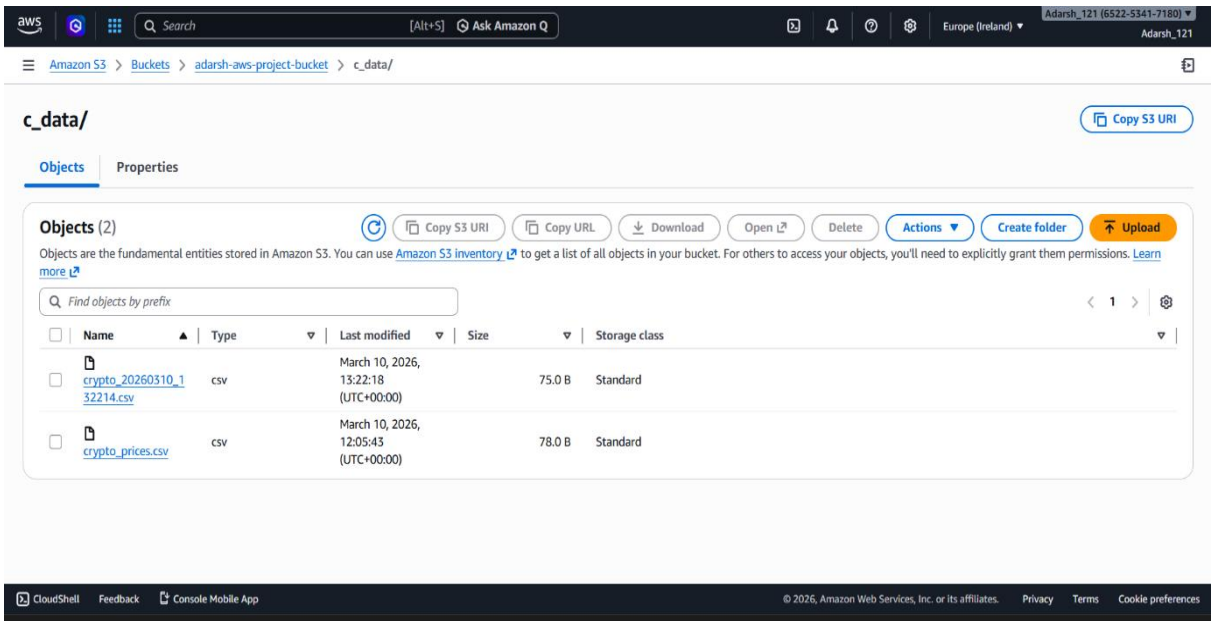
*SELECT * FROM "crypto_db.crypto_data_new";*



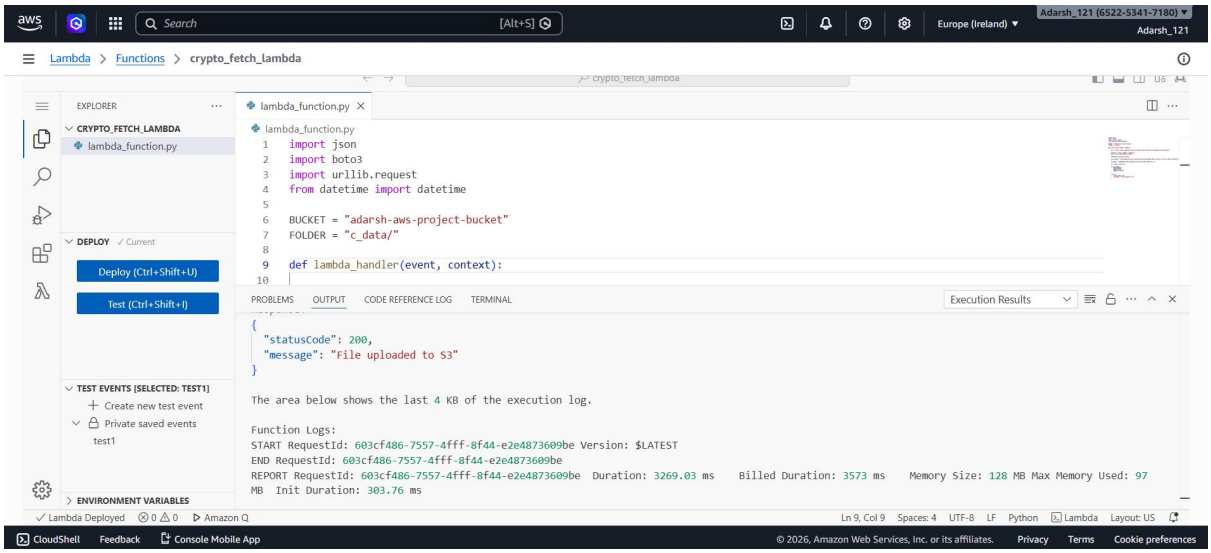
7. Challenges & Troubleshooting

Working with cloud infrastructure naturally introduces configuration and execution challenges. Key issues resolved during this project include:

- **Athena Returning NULL Rows:**
  - *Cause:* Initially, the Athena table referenced the root of the S3 bucket. Because Athena scans everything in the target path, it attempted to read non-data metadata files, resulting in NULL rows in the output.
  - *Solution:* Created a dedicated c_data/ directory exclusively for the CSV files and repointed the Athena table to only scan that specific folder.



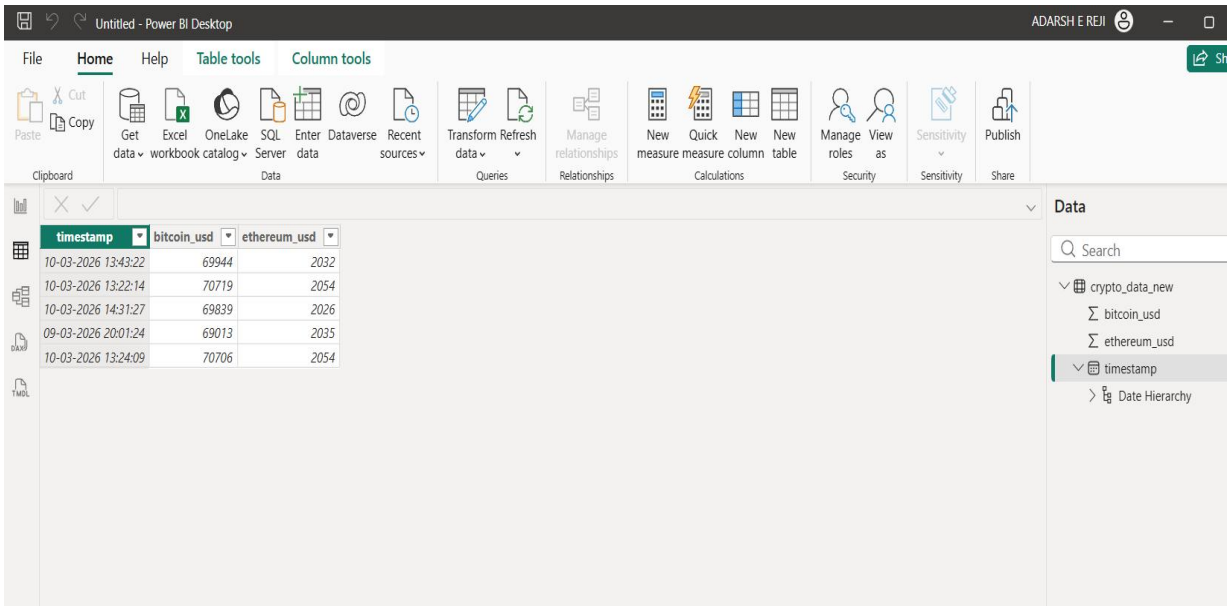
- **AWS Lambda Execution Timeout:**
    - *Cause:* The Lambda function failed with a Task timed out after 3 seconds error. The default 3-second timeout was too short to complete the external API handshake and S3 upload.
    - *Solution:* Increased the Lambda configuration timeout to 10 seconds, allowing the function to execute reliably.

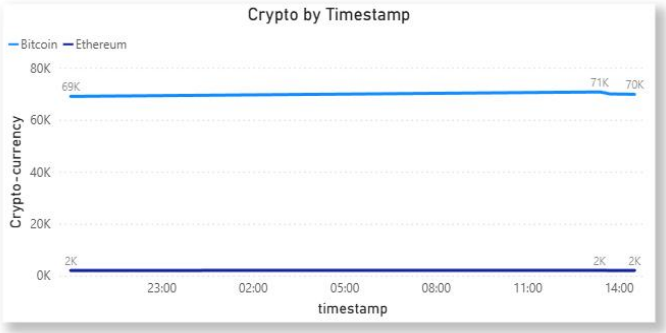


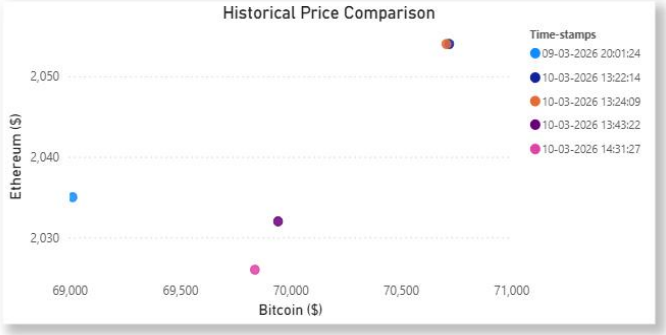
8. Data Visualization (Power BI)

To make the data actionable, Power BI was connected to Amazon Athena. This was achieved using an ODBC connection authenticated via AWS IAM credentials, ensuring a secure read-only pipeline from the cloud to the BI tool.

The resulting dashboard visualizes the price fluctuations of Bitcoin and Ethereum over time, automatically updating as new data drops into the S3 bucket.







9. Future Improvements

To scale this system into an enterprise-grade pipeline, future iterations could include:

- **Automation:** Using Amazon EventBridge to schedule the Lambda function to run automatically (e.g., every 5 minutes).
- **Data Transformation:** Implementing AWS Glue for more complex ETL (Extract, Transform, Load) operations before querying.
- **Data Warehousing:** Moving larger, historical datasets into Amazon Redshift for faster, large-scale analytics.
- **Cloud-Native BI:** Building real-time, cloud-hosted dashboards using Amazon QuickSight instead of local Power BI instances.

10. Conclusion

This project successfully demonstrates how to architect a lightweight, functional cloud data pipeline using AWS serverless tools. By handling API ingestion, scalable storage, and secure BI integration, it serves as a practical foundation for understanding how modern data architectures operate at scale.